UNIVERSIDAD TÉCNICA FEDERICO SANTA MARÍA

ICN292

Laboratorio 5

Consultas SQL sobre la base andesbike

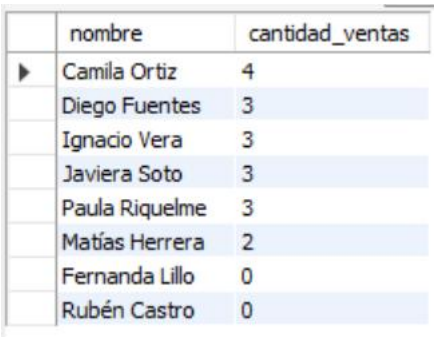
Estudiante: Montserratt Rojas
Rol USM: 202260583-9
Fecha: 11 de noviembre de 2026
Repositorio: https://github.com/MontseR55/ICN292-Lab5-Rojas-Montserratt

Consulta 1

Código SQL

```
SELECT v.nombre,
       COUNT(ve.id_venta) AS cantidad_ventas
FROM vendedores v
LEFT JOIN ventas ve ON ve.id_vendedor = v.id_vendedor
GROUP BY v.id_vendedor, v.nombre
ORDER BY cantidad_ventas DESC, v.nombre;
```

Resultado en MySQL Workbench



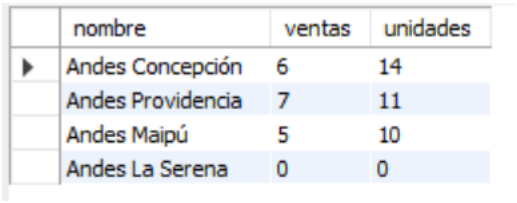
nombre	cantidad_ventas
Camila Ortiz	4
Diego Fuentes	3
Ignacio Vera	3
Javiera Soto	3
Paula Riquelme	3
Matías Herrera	2
Fernanda Lillo	0
Rubén Castro	0

Consulta 2

Código SQL

```
SELECT t.nombre,
       COUNT(ve.id_venta)           AS ventas,
       COALESCE(SUM(ve.unidades),0) AS unidades
FROM tiendas t
LEFT JOIN vendedores v  ON v.id_tienda = t.id_tienda
LEFT JOIN ventas ve     ON ve.id_vendedor = v.id_vendedor
GROUP BY t.id_tienda, t.nombre
ORDER BY unidades DESC;
```

Resultado en MySQL Workbench



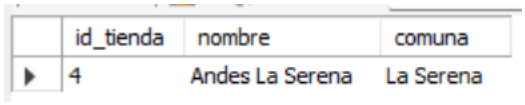
nombre	ventas	unidades
Andes Concepción	6	14
Andes Providencia	7	11
Andes Maipú	5	10
Andes La Serena	0	0

Consulta 3

Código SQL

```
SELECT t.id_tienda, t.nombre, t.comuna
FROM tiendas t
LEFT JOIN vendedores v ON v.id_tienda = t.id_tienda
WHERE v.id_vendedor IS NULL;
```

Resultado en MySQL Workbench



	id_tienda	nombre	comuna
▶	4	Andes La Serena	La Serena

Consulta 4

Código SQL

```
SELECT b.modelo
FROM bicicletas b
LEFT JOIN ventas ve
       ON ve.id_bicicleta = b.id_bicicleta
      AND ve.fecha >= '2025-06-01'
      AND ve.fecha <  '2025-07-01'
WHERE ve.id_venta IS NULL;
```

Resultado en MySQL Workbench



	modelo
▶	Ruta Austral
	Urbana Pro
	Volt City
	Sprint R7

Explicación

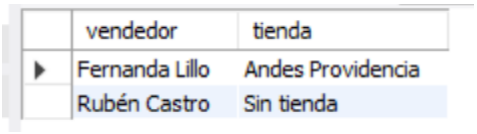
Puse la condición del mes en el ON del LEFT JOIN (junto al cruce por id_bicicleta); si la hubiera puesto en el WHERE, la consulta no habría devuelto ninguna fila, porque el WHERE exigiría una venta de junio y a la vez que no existiera venta.

Consulta 5

Código SQL

```
SELECT v.nombre AS vendedor,
       COALESCE(t.nombre, 'Sin tienda') AS tienda
FROM vendedores v
LEFT JOIN ventas ve ON ve.id_vendedor = v.id_vendedor
LEFT JOIN tiendas t ON t.id_tienda = v.id_tienda
WHERE ve.id_venta IS NULL;
```

Resultado en MySQL Workbench



vendedor	tienda
Fernanda Lillo	Andes Providencia
Rubén Castro	Sin tienda

Consulta 6

Código SQL

```
SELECT v1.nombre        AS vendedor_antiguo,
       v1.fecha_ingreso AS ingreso_antiguo,
       v2.nombre        AS vendedor_reciente,
       v2.fecha_ingreso AS ingreso_reciente,
       t.nombre         AS tienda
FROM vendedores v1
JOIN vendedores v2
  ON v1.id_tienda = v2.id_tienda
 AND (v1.fecha_ingreso < v2.fecha_ingreso
      OR (v1.fecha_ingreso = v2.fecha_ingreso AND v1.id_vendedor < v2.id_vendedor))
JOIN tiendas t ON t.id_tienda = v1.id_tienda
ORDER BY t.nombre, v1.fecha_ingreso, v2.fecha_ingreso;
```

Resultado en MySQL Workbench

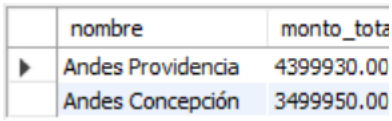
vendedor_antiguo	ingreso_antiguo	vendedor_reciente	ingreso_reciente	tienda
Paula Riquelme	2022-11-02	Ignacio Vera	2024-08-19	Andes Concepción
Javiera Soto	2023-07-24	Matías Herrera	2024-01-15	Andes Maipú
Diego Fuentes	2024-01-15	Camila Ortiz	2024-06-20	Andes Providencia
Diego Fuentes	2024-01-15	Fernanda Lillo	2025-02-03	Andes Providencia
Camila Ortiz	2024-06-20	Fernanda Lillo	2025-02-03	Andes Providencia

Consulta 7

Código SQL

```
SELECT t.nombre,
       SUM(ve.unidades * b.precio) AS monto_total
FROM tiendas t
JOIN vendedores v ON v.id_tienda = t.id_tienda
JOIN ventas ve    ON ve.id_vendedor = v.id_vendedor
JOIN bicicletas b ON b.id_bicicleta = ve.id_bicicleta
WHERE b.categoria IN ('Montaña', 'Ruta')
GROUP BY t.id_tienda, t.nombre
HAVING SUM(ve.unidades * b.precio) > 2000000
ORDER BY monto_total DESC;
```

Resultado en MySQL Workbench



	nombre	monto_tota
▶	Andes Providencia	4399930.00
	Andes Concepción	3499950.00

Consulta 8

Código SQL

```
SELECT modelo, precio
FROM bicicletas
WHERE precio > (SELECT AVG(precio) FROM bicicletas)
ORDER BY precio DESC;
```

Resultado en MySQL Workbench



modelo	precio
Volt City	1249990.00
Sprint R7	1099990.00
Ruta Austral	899990.00
Trail Master	749990.00

Explicación

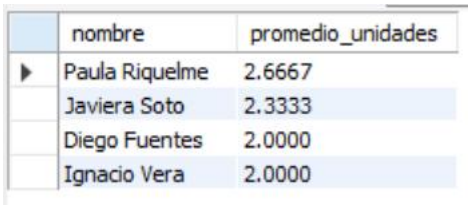
El promedio del catálogo es 657.490, y lo obtuve con la subconsulta (SELECT AVG(precio) FROM bicicletas), es decir, desde la tabla bicicletas y no desde las ventas.

Consulta 9

Código SQL

```
SELECT v.nombre,
       AVG(ve.unidades) AS promedio_unidades
FROM vendedores v
JOIN ventas ve ON ve.id_vendedor = v.id_vendedor
GROUP BY v.id_vendedor, v.nombre
HAVING AVG(ve.unidades) > (SELECT AVG(unidades) FROM ventas)
ORDER BY promedio_unidades DESC, v.nombre;
```

Resultado en MySQL Workbench



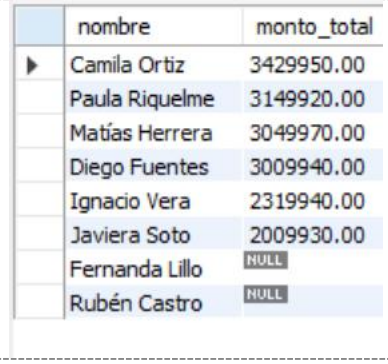
nombre	promedio_unidades
Paula Riquelme	2.6667
Javiera Soto	2.3333
Diego Fuentes	2.0000
Ignacio Vera	2.0000

Consulta 10

Código SQL

```
SELECT v.nombre,
       SUM(ve.unidades * b.precio) AS monto_total
FROM vendedores v
LEFT JOIN ventas ve    ON ve.id_vendedor = v.id_vendedor
LEFT JOIN bicicletas b ON b.id_bicicleta = ve.id_bicicleta
GROUP BY v.id_vendedor, v.nombre
ORDER BY monto_total DESC;
```

Resultado en MySQL Workbench



nombre	monto_total
Camila Ortiz	3429950.00
Paula Riquelme	3149920.00
Matías Herrera	3049970.00
Diego Fuentes	3009940.00
Ignacio Vera	2319940.00
Javiera Soto	2009930.00
Fernanda Lillo	NULL
Rubén Castro	NULL

Explicación

Fernanda Lillo y Rubén Castro muestran NULL porque SUM sobre un conjunto vacío devuelve NULL y no 0; para que muestren cero habría que agregar COALESCE(SUM(ve.unidades * b.precio), 0) en el SELECT.